Statistical Machine Learning Assignment

Name: Om Tripathi
Roll Number: BA2025027

Q1: Decision Trees

(a) Entropy, Gini, and Information Gain Total samples: 200
Class 1: 120, Class 0: 80
Entropy at root:

$$H = -p_1 \log_2 p_1 - p_0 \log_2 p_0$$

$$= -(0.6 \log_2 0.6 + 0.4 \log_2 0.4) = 0.971$$

Gini impurity:

$$G = 1 - (p_1^2 + p_0^2) = 1 - (0.6^2 + 0.4^2) = 0.48$$

After split:
Left node entropy:

$$H_L = 0.764$$

Right node entropy:

$$H_R = 0.994$$

Weighted entropy:

$$H_{split} = \frac{90}{200}(0.764) + \frac{110}{200}(0.994) = 0.891$$

Information Gain:

$$IG = 0.971 - 0.891 = 0.08$$

(b) ID3 vs CART ID3 selects the split with the highest information gain (entropy reduction), while CART selects the split that minimizes Gini impurity.
If a feature has higher information gain but lower Gini reduction:

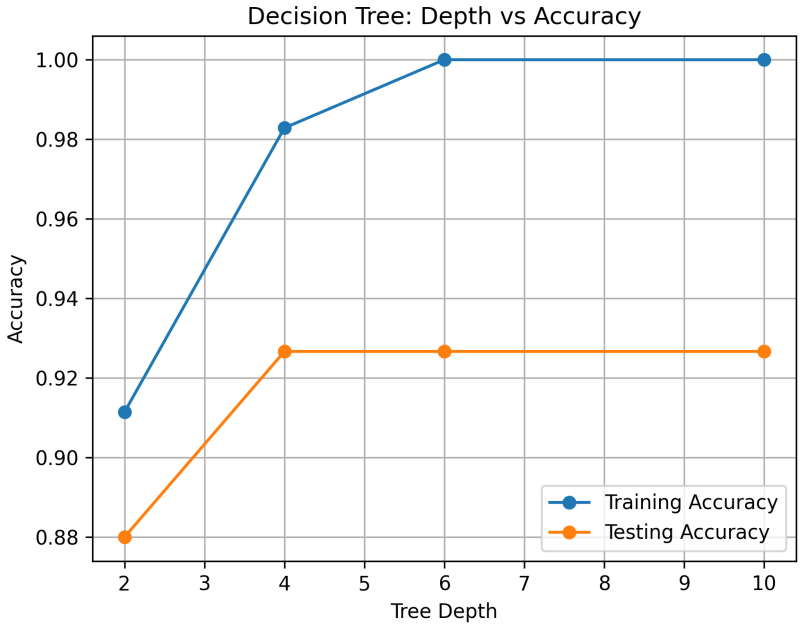
- ID3 will choose that feature.
- CART will choose the feature with lower Gini impurity.

The algorithms may produce different trees because they use different impurity measures. Entropy is more sensitive to class probabilities, while Gini is computationally simpler and focuses on classification purity.

(c) Overfitting and Bias-Variance Tradeoff A fully grown tree perfectly fits training data but performs poorly on test data due to high variance.

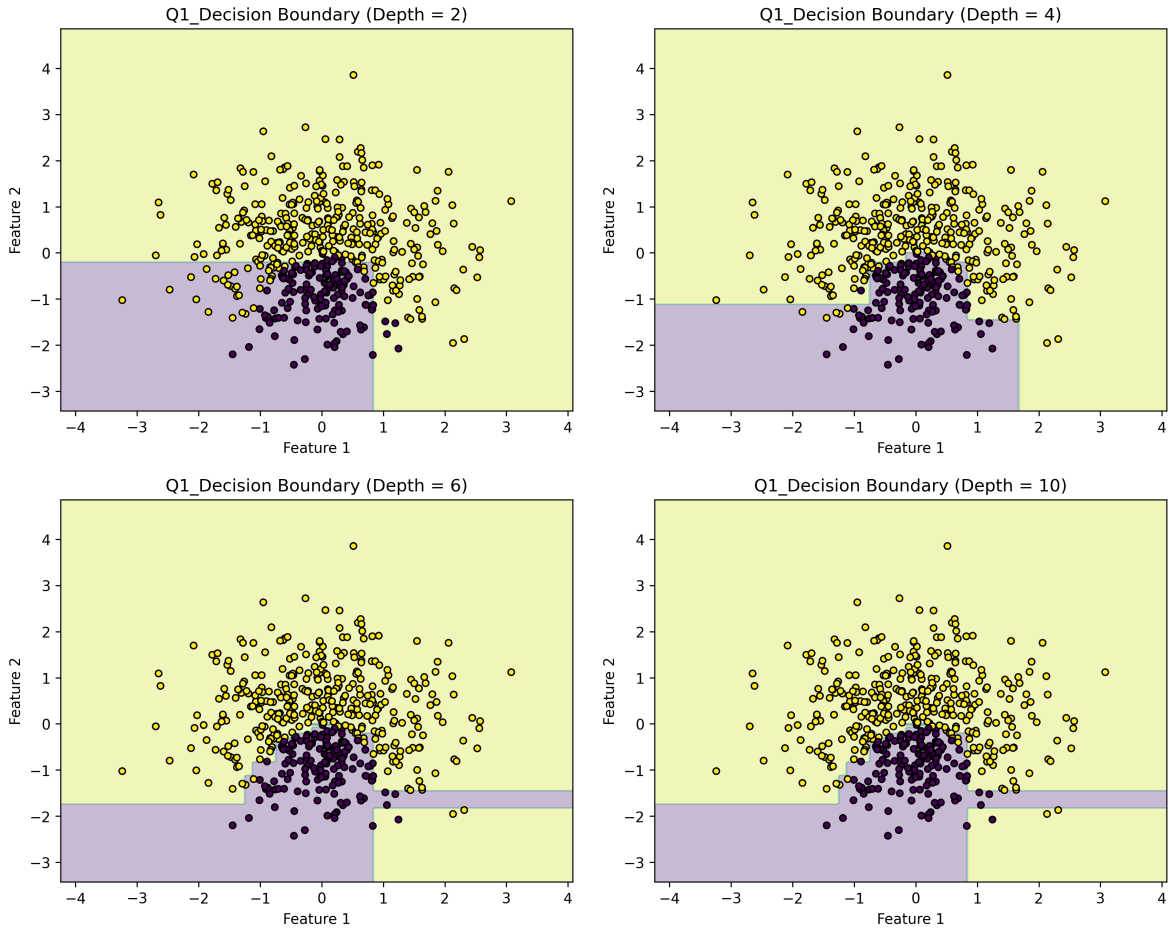
- **Max depth:** Larger depth increases variance and overfitting.
- **Min samples split:** Higher values prevent splitting on small noisy data.
- **Post-pruning:** Removes unnecessary branches and reduces overfitting.

(d) Experiment: Effect of Tree Depth A synthetic dataset with 2 informative and 3 noisy features was generated. Decision trees were trained with depths 2, 4, 6, and 10.
Accuracy vs Depth:



Training accuracy increases with depth, while testing accuracy initially improves and then decreases, indicating overfitting at higher depths.

Decision Boundaries:



Shallow trees produce simple decision boundaries and underfit the data. As depth increases, the boundary becomes more flexible and better captures the structure of the data. At higher depths, the boundary becomes highly irregular, indicating overfitting.

Q2: Logistic Regression

(a) Log-Likelihood and Gradients The probability model is:

$$P(Y = 1|x) = \frac{1}{1 + e^{-(wx+b)}}$$

The log-likelihood function is:

$$L(w, b) = \sum_{i=1}^{n} [y_i \log(\sigma(z_i)) + (1 - y_i) \log(1 - \sigma(z_i))]$$

where $z_i = wx_i + b$.
The gradients are:

$$\frac{\partial L}{\partial w} = \sum (y_i - \hat{y}_i) x_i$$

$$\frac{\partial L}{\partial b} = \sum (y_i - \hat{y}_i)$$

where $\hat{y}_i = \sigma(wx_i + b)$.

(b) Numerical Computation Given $w = 0.5$, $b = -1$:
Predicted probabilities:

$$\hat{y} = [0.377, 0.5, 0.622, 0.731]$$

Cross-entropy loss:

$$L \approx 2.79$$

Gradients:

$$\frac{\partial L}{\partial w} \approx -1.665$$

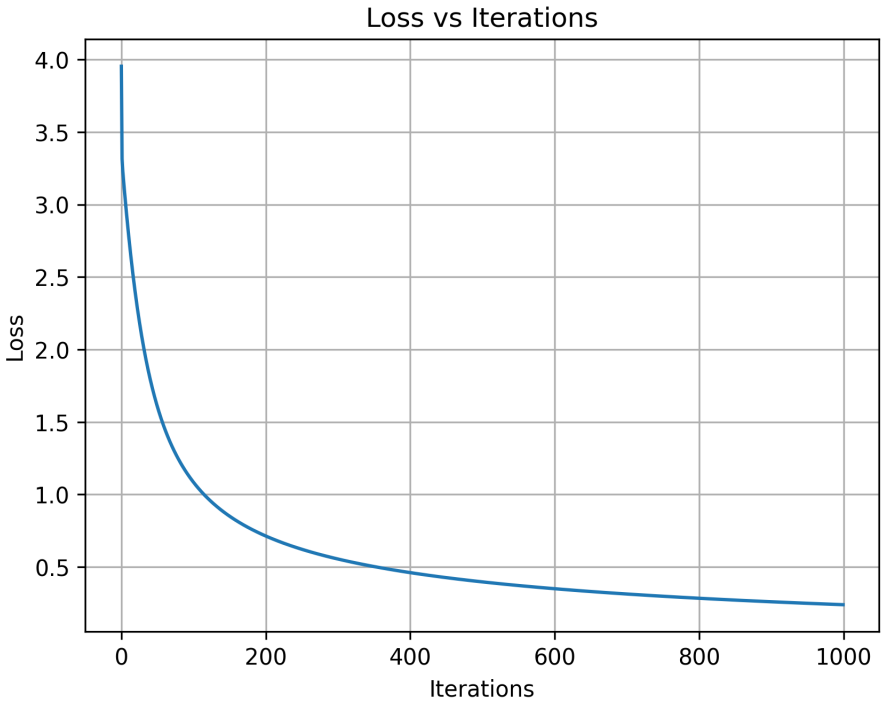
$$\frac{\partial L}{\partial b} \approx -0.23$$

(c) Linear Decision Boundary Although logistic regression uses a nonlinear sigmoid function, the decision boundary is linear because it is defined by:

$$wx + b = 0$$

This represents a straight line in the feature space. The sigmoid function only maps the linear combination into probabilities.

(d) Implementation using Gradient Descent Logistic regression was implemented from scratch using gradient descent.
The model was trained for 1000 iterations with learning rate 0.1. The loss decreased monotonically, indicating successful convergence.



The parameters obtained from the custom implementation were close to those obtained using `sklearn.linear_model.LogisticRegression`, validating the correctness of the implementation.

Q3: Polynomial Regression

(a) Normal Equation and Design Matrix The model is:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2$$

The normal equation is:

$$\beta = (X^T X)^{-1} X^T y$$

The design matrix is:

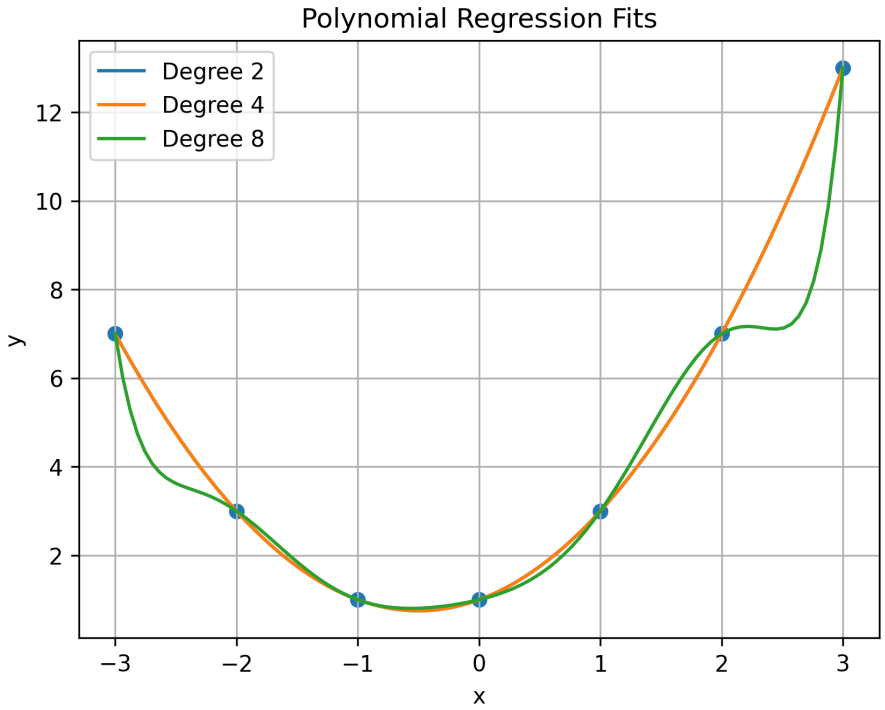
$$X = \begin{bmatrix} 1 & -3 & 9 \\ 1 & -2 & 4 \\ 1 & -1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \end{bmatrix}$$

(b) Effect of Polynomial Degree Higher-degree polynomial models can overfit the data by capturing noise instead of the true underlying pattern.
As the degree increases:

- Variance increases
- Bias decreases

This leads to overfitting at higher degrees.
Although polynomial regression appears nonlinear, it is linear in parameters since the model is linear in $\beta_0, \beta_1, \beta_2, \ldots$.

(c) Polynomial Regression Experiment Polynomial models of degrees 2, 4, and 8 were fitted.



It was observed that higher-degree polynomials fit the training data better but may overfit, resulting in highly oscillatory curves.

(d) Ridge Regularization The ridge regression solution is:

$$\beta = (X^T X + \lambda I)^{-1} X^T y$$

As λ increases:

- Bias increases
- Variance decreases

This helps prevent overfitting by shrinking the coefficients.

Q4: K-Means Clustering

(a) One Iteration Initial centroids:

$$C_1 = (1, 1), \quad C_2 = (5, 7)$$

After computing distances, the clusters are:
Cluster 1: $(1, 1), (1.5, 2)$
Cluster 2: $(3, 4), (5, 7), (3.5, 5), (4.5, 5), (3.5, 4.5)$
Updated centroids:

$$C_1 = (1.25, 1.5)$$

$$C_2 = (3.9, 5.1)$$

(b) Objective Function The K-means objective is:

$$J = \sum_{i=1}^{K} \sum_{x \in C_i} \|x - \mu_i\|^2$$

Each iteration consists of assignment and update steps, both of which reduce the objective function. Therefore, the value of J never increases.

(c) Local Minima and Initialization K-means may converge to a local minimum depending on initialization.
Random initialization may lead to poor clustering, while K-means++ improves initialization by spreading centroids apart, leading to better convergence.

(d) Implementation K-means was implemented from scratch and run with different initializations.
Different runs resulted in different cluster assignments and SSE values, demonstrating sensitivity to initialization and convergence to local minima.